

Advanced / Custom RAG Engineering

- Design production-grade RAG architectures.
- Implement hybrid retrieval and metadata filtering strategies.
- Optimize context usage and retrieval performance.
- Build enterprise-level knowledge retrieval systems.
- Pinecone
- Chroma
- Hybrid Search (BM25 + Vector)
- Re-ranking models
- Hybrid retrieval strategies
- Metadata filtering
- Query rewriting
- Context window optimization
- Structured + unstructured RAG
- Multi-source ingestion
- RAG over APIs
- Embedding optimization
- Retrieval performance tuning

Hybrid Search (BM25 + Vector Search)

What is Hybrid Search?

Hybrid Search is a retrieval technique that combines **Keyword Search (BM25)** and **Semantic Search (Vector Search)** to retrieve the most relevant documents. Instead of relying on only exact keyword matching or only on semantic similarity, Hybrid Search uses both methods together to improve search accuracy.

Definition

Hybrid Search is the process of combining sparse retrieval (BM25) and dense retrieval (vector embeddings) to return more accurate and relevant search results.

Why Do We Need Hybrid Search?

No single search technique is perfect.

1. Keyword Search (BM25)-best matching

BM25 searches for exact words present in the documents.

Example:

Documents

1. Pinecone is a vector database.

2. BM25 is a keyword ranking algorithm.

Query:

What is BM25?

BM25 finds the exact word "BM25", so it retrieves:

BM25 is a keyword ranking algorithm.

This is correct.

Problem with BM25

Suppose the user searches:

Semantic Search Database

Document:

Pinecone is a vector database.

Since the document doesn't contain the exact words Semantic Search, BM25 may fail to retrieve it.

2. Vector Search

Vector Search converts text into embeddings (vectors) and searches based on meaning, not exact words.

Example:

Document:

Pinecone is a vector database.

Query:

Semantic Search Database

Even though the words are different, the meanings are similar.

Vector Search understands:

Semantic Search Database

=

Vector Database

So it retrieves the correct document.

Problem with Vector Search

Suppose the query is:

ERROR

Document:

Error Code E88500 occurs due to authentication failure.

Vector Search may not prioritize this document because E88500 is a specific keyword, not a semantic concept.

BM25 performs much better in such cases.

Solution: Hybrid Search

Hybrid Search combines the strengths of both approaches.

User Query

Hybrid Search

BM25

Vector Search

Merge Scores

Top Relevant Documents

Comparison

Feature

Exact keyword matching

Semantic understanding

Handles synonyms

Product ID's / Error Codes

Natural language queries

Overall accuracy

Medium

High

Very High

Hybrid

Who decides the BM25 and Vector scores?

The BM25 algorithm automatically adjusts the keyword relevance based on term frequency, inverse document frequency, and document length.

The vector database automatically computes the semantic similarity scores using a similarity metric such as cosine similarity between the query embedding and document embeddings.

The developer decides how to combine these scores (for example, by choosing the alpha weight), and in production this weight is typically refined through experimentation or replaced with more advanced ranking techniques like RRF or learning-to-rank models.

What is Re-ranking?

Definition

Re-ranking is the process of re-evaluating the retrieved documents using a more powerful model to improve their order before sending them to the LLM.

Think of it as a second round of judging.

Example

Support the user asks:

How does hybrid search work?

After hybrid search, we get:

Rank

Document

Hybrid Score

1. Pinecone supports hybrid search. 0.95

2. BM25 is a keyword ranking algorithm. 0.81

3. Vector databases store embeddings. 0.88

4. Elasticsearch supports BM25. 0.85

5. Pinecone pricing details. 0.82

Hybrid Search says:

"These are probably the best documents."

But are they really?

Re-ranking Model

A re-ranker reads:

User Question

AND

Each document together

instead of looking at vectors alone.

Example:

Question:

How does Hybrid Search work?

Document 1

Score = 0.82

Question:

How does Hybrid Search work?

Document 2

Score = 0.95

Question:

How does Hybrid Search work?

Document 3

Score = 0.45

Now the ranking changes.

Before

After Re-ranking

Pinecone supports Hybrid Search

BM25 is a keyword algorithm

Pinecone supports hybrid search

Vector databases store embeddings

Elasticsearch supports BM25

The order becomes better.

Why is Re-ranking Needed?

Vector Search only compares embeddings.

It doesn't deeply understand the question.

Example:

Query:

How does Hybrid Search work?

Document A

Hybrid Search

Hybrid Search combines BM25 and Vector Search.

Document B

Pinecone pricing for Hybrid Search.

Both contain:

Hybrid Search

Vector Search may rank them similarly.

A re-ranker reads the entire question and document together, realizing that Document A actually explains how it works, while Document B is about pricing.

Retrieval Pipeline

Documents

Vector DB / BM25

Top 20 Documents

Re-ranking Model

Imagine a Class of 5 Students

Imagine everyone to write an essay.

Document 1

AI Machine Learning

Document 2

Machine Learning Python

Document 3

Cricket Football Tennis

Document 4

AI Machine Python

Document 5

Cooking Recipes Food

Now someone searches for:

Step 1: Term Frequency (TF)

Ask students:

How many times does "AI" appear inside each document?

Document

AI Count (TF)

Document 1

Document 2

Document 3

Document 4

Document 5

0

1

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

Imagine a Class of 5 Students

Imagine everyone to write an essay.

Document 1

AI Machine Learning

Document 2

Machine Learning Python

Document 3

Cricket Football Tennis

Document 4

AI Machine Python

Document 5

Cooking Recipes Food

Now someone searches for:

Step 1: Term Frequency (TF)

Ask students:

How many times does "AI" appear inside each document?

Document

AI Count (TF)

Document 1

Document 2

Document 3

Document 4

Document 5

0

1

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0



Retrieval Pipeline



What Models Are Used for Re-ranking?

The most common are Cross-Encoder models. Unlike embedding models, which encode the query and document separately, a Cross-Encoder processes them together.

Query
+
Document
↓
Transformer
↓
Relevance Score

Popular Re-ranking Models

1. BAAI/bge-reranker-large

Very popular.
From sentence_transformers import CrossEncoder
model = CrossEncoder("BAAI/bge-reranker-large")
Example:
score = model.predict(["What is hybrid search?", "Hybrid search combines BM25 and Vector Search."])
Output:
0.98

2. BAAI/bge-reranker-base

Smaller and faster.
Good for local applications.

3. Cohere Rerank API

Cloud API.
Query
↓
Cohere
↓
Top documents reordered
Very common in enterprise RAG.

Difference Between Embedding Model and Re-ranking Model

Embedding Model	Re-ranking Model
Encodes documents once and stores vectors	Reads the query and document together at search time
Very fast retrieval	Slower but more accurate
Finds similar documents	Reorders retrieved documents
Used before retrieval	Used after retrieval

Real Production Flow



Metadata Filtering

What is Metadata Filtering?

Metadata filtering is a retrieval technique where documents are filtered based on their metadata (properties) before or during retrieval.

Instead of searching all documents, we first select only the documents that satisfy specific conditions.

Definition
Metadata filtering restricts the search space by applying conditions on document metadata such as author, date, category, language, source, department, or location before performing retrieval.

What is Metadata?

Metadata is data about the document, not the document content.

Example document:

Document:
Pinecone is a vector database.
Metadata:
"id": "101",
"title": "Introduction to Pinecone",
"author": "John Doe",
"category": "AI",
"language": "English",
"year": 2023,
"department": "Engineering".
Note:
The metadata is not embedded.
Only the document text is embedded.

Why Do We Need Metadata Filtering?

Imagine your vector database contains 10 million documents.

Without filtering:
User Query
↓
Search all 10 million documents
Very expensive.

Suppose the user asks:
Show AI documents published in 2023
Instead of searching all documents

We first filter:
Category: AI
Year: 2023
Now only
10,000 documents
remain.

Vector Search runs only on those documents.
Much faster.

Architecture

User Query
↓
Metadata Filter
(Category: AI, Year: 2023)
↓
Filtered Documents
↓
Vector Search
↓
Top Documents
↓
LLM

Example

Suppose we have

1 Document
2 Pinecone Introduction AI 2023
3 Python Basics Programming 2024
4 LangChain Tutorial AI 2023
5 React Guide Web 2023

User asks:
LangChain
Without metadata filter
Vector Search checks
1
2
3
4
5

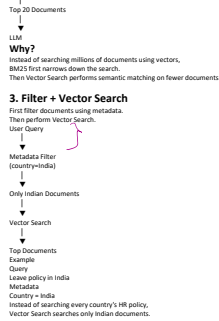
Context Window Optimization

What is Context Window Optimization?

Definition
Context Window Optimization is the process of selecting, compressing, and organizing the most relevant information so that it fits within an LLM's context window while maintaining answer quality.

Simply put:
Instead of sending all retrieved documents to the LLM, we send only the most useful information.

Why Do We Need It?

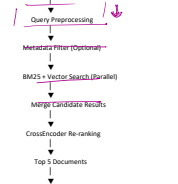


Retrieval Strategy Comparison

Strategy	Best Use Case
Parallel Hybrid: BM25 + Vector Together	General RAG systems
Sequential: BM25 → Vector	Large document collections
Filter → Vector: Metadata → Vector	Enterprise search
Multi-stage: BM25 → Vector → Re-ranker	Production RAG
Query Routing: Choose retriever based on query	Mixed workloads

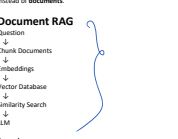
Which Strategy is Used in Production?

Most enterprise systems use a multi-stage hybrid pipeline.



RAG over APIs

What is API RAG?
API RAG is a type of Retrieval-Augmented Generation where the LLM retrieves information from APIs instead of documents.



API RAG



Example:
What is my leave balance?
GET /leave/bal:
"leave_balance": 18
LLM:
You have 18 leave days remaining.

Why No Chunking or Embeddings?

Because the API already returns the exact data.

- No Chunk documents
- No Create embeddings
- No Store vectors
- No Perform similarity search

What Does the LLM Add?

- Understands the user's intent.
- Chooses the correct API.
- Converts JSON into a natural language response.
- Can combine data from multiple APIs.

Example

User:
Can I take leave tomorrow?
LLM calls:
• Leave API
• Holiday API
Combines both responses and answers:
"Yes, you have 18 leave days remaining, and tomorrow is a working day."

API RAG vs Document RAG

Document RAG	API RAG
PDFS, Docs	REST/GraphQL APIs
Chunking	No Chunking
Embeddings	No Embeddings
Vector DB	No Vector DB
Static Data	Live Data

Key Takeaway

RAG is not equal to Vector Database.

RAG = Retrieve external information + LLM generating the answer.

The retrieval source can be:
• Documents (PDFs, Word)
• APIs
• SQL Databases
• Graph Databases
• Elasticsearch

A vector database is just one way to perform retrieval, not the definition of RAG.

1 I want to know leave?

2 AHEAD call first

Attend (API, B09)

leave doc

11111

11111

11111

11111

11111

11111

11111

11111

11111

11111

11111

11111

11111

11111

11111

11111

11111

11111

11111

11111

11111

11111

11111

11111

11111

11111

11111

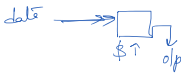
11111

What is Context Window Optimization?

Definition

Context Window Optimization is the process of selecting, compressing, and organizing the most relevant information so that it fits within an LLM's context window while maintaining answer quality.

Simply put:
Instead of sending all retrieved documents to the LLM, we send only the most useful information.



Why Do We Need It?

Every LLM has a context window (maximum number of tokens it can process).

Example:

- Model Context Window
- GPT-4.1 Large context window
 - Gemini 2.5 Large context window
 - Claude Large context window
- Even with large context windows, sending unnecessary information:
- Increases cost
 - Increases latency
 - Can confuse the model
 - Reduces answer quality
- Handwritten note: data (irrelevant)

Example

Suppose your Vector DB returns

- Document 1 → 1500 tokens
- Document 2 → 1500 tokens
- Document 3 → 1200 tokens
- Document 4 → 950 tokens
- Document 5 → 900 tokens

Total: 5400 tokens
But only 1200 tokens are actually useful.
Instead of sending 5400 tokens, we optimize and send 1200 tokens.

Without Optimization



With Context Window Optimization



Common Context Window Optimization Techniques

1. Top-K Retrieval

Instead of sending 20 documents, send top 3 documents.

Example:
results = index.query(
vector=query_embedding,
top_k=3
)

2. Re-ranking

Hybrid Search returns 20 documents.
Re-ranker selects top 5.
Only those go to the LLM.

3. Chunk Selection

Suppose a document has 100 paragraphs.
The answer exists only in Paragraph 62.
Instead of sending 100 paragraphs, send only Paragraph 62.

4. Context Compression

Original:
The annual leave policy was introduced in 2022 (long explanation).
Employees receive 20 days of leave.
Many holidays...
Compressed:
Employees receive 20 days of annual leave.
The important information remains.

5. Remove Duplicate Chunks

Retrieved:
Chunk 1
Pinecone is a vector database.
Chunk 2
Pinecone is a vector database.
Chunk 3
Pinecone stores embeddings.
Instead of sending:
1
2
3
Remove duplicates.
Send:
1
3

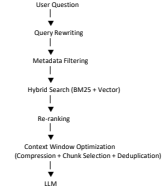
6. Metadata Filtering

Instead of searching 10 Million Docs, search only HR Documents.
HR Documents: Less context, Better retrieval.

7. Query Rewriting

Original: How does Pinecone work?
Rewrite: How does Pinecone Hybrid Search work?
Better retrieval.
Less unnecessary content.

Production RAG Pipeline



	doc	Similarity	Why?
FAQ's	C.S 100-200	C.O 20-50	1-2 lines (But context already)
HR Policy	300-600	50-100	Longer Paragraph
Research paper	↑ ↑	↑ ↑	—
Legal contract	+		
Source code	many lines of code	2	Logical unit

100 PDF
P → 1
2
⋮
P → 100
What is Pinecone?
LLM → 100 PDFs
Rerank

Chunk
Search doc
Question ↔ Chunk
15 →
20 → Pinecone
62 →
88 →

Similarity

	Similar
15	0.21
20	0.80
62	0.18
88	0.11

Embedding Optimization

What is Embedding Optimization?

Definition

Embedding Optimization is the process of improving how text is converted into vector embeddings so that semantically similar documents are retrieved more accurately and efficiently.

In simple words:
Instead of simply calling embedding = model.encode(text), we optimize what we embed, how we embed it, and which model we use to improve retrieval quality.

Why Do We Need Embedding Optimization?

Suppose you have two documents:
Doc1:
Pinecone is a vector database.
Doc2:
Apple released iPhone 16.
User asks:
What is a vector DB?
If our embeddings are poor, the retriever might return Apple released iPhone 16.
Instead of Pinecone is a vector database.
Embedding optimization improves retrieval accuracy.

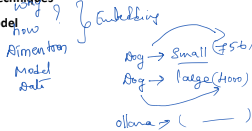
Where does it happen?



Common Embedding Optimization Techniques

1. Choosing the Right Embedding Model

★★★★★
The biggest optimization!
Poor Model
SentenceTransformer("all-MiniLM-L6-v2")
Better Model
text-embedding-3-small
Even Better
text-embedding-3-large
Different models produce different quality embeddings.



Example

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("all-MiniLM-L6-v2")

embedding = model.encode("What is Hybrid Search?")

# Changing the model changes retrieval quality.
```



2. Clean the Text Before Embedding

Bad
Pinecone#### is a vector database!!!!
Clean
Pinecone is a vector database.
Cleaner text usually leads to better embeddings.
Example
import re
text = "Pinecone#### is a vector database!!!!"
text = re.sub(r"[^a-zA-Z0-9]+", "", text)
print(text)
Output
Pinecone is a vector database



3. Chunk Optimization

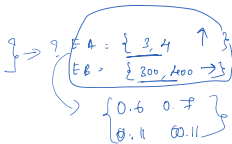
Instead of embedding a huge document
5000 words
Split it into chunks.
Example
from langchain.text_splitter import RecursiveCharacterTextSplitter
splitter = RecursiveCharacterTextSplitter(
 chunk_size=500,
 chunk_overlap=100
)
chunks = splitter.split_text(document)
Then
embeddings = model.encode(chunks)
Better retrieval.

4. Add Metadata Before Embedding

Instead of embedding only
Leave Policy
Embed
Department: HR
Leave Policy
Now the embedding contains more context.
Example
text = ""
Department: HR
Leave Policy:
Employees receive 20 days leave.
""
Better semantic representation.

5. Query Optimization

Document
Hybrid Search combines BM25 and Vector Search.
User
Hybrid
Rewrite
What is Hybrid Search?
Generates embedding from rewritten query.
Better retrieval.



6. Normalize Embeddings

Many vector databases expect normalized vectors.
embedding = embedding / np.linalg.norm(embedding)
Now every vector has unit length.
This improves cosine similarity calculations.

7. Domain-Specific Embeddings

Medical documents
↓
Medical embedding model
Legal documents
↓
Legal embedding model
Code
↓
Code embedding model
Using a domain-specific model often improves retrieval.

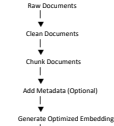
8. Remove Noise

Don't embed
Footer
Page Number
Copyright
Contact
Only embed
Actual Content
Noise reduces embedding quality.

9. Cache Embeddings

Don't generate embeddings every time.
Store them.
embedding_cache[text] = embedding
Huge performance improvement.

Production Flow



Retrieval Performance Tuning

What is Retrieval Performance Tuning?

Definition

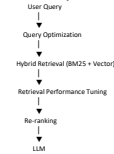
Retrieval Performance Tuning is the process of improving the accuracy, speed, relevance, and efficiency of the document retrieval stage in a RAG system.

The goal is to retrieve:
• More relevant documents
• Faster results
• Fewer irrelevant documents
• Lower latency and cost

Why is it Important?

Imagine you have 10 million documents.
User asks:
How does Hybrid Search work?
If the retriever returns:
1. Apple iPhone Launch
2. Leave Policy
1. Hybrid Search combines BM25 and Vector Search
The LLM may generate a poor answer because only one retrieved document is relevant.
Retrieval performance tuning helps retrieve:
1. Hybrid Search combines BM25 and Vector Search
2. BM25 Keyword Search
1. Vector Search
Now the LLM has much better context.

Retrieval Pipeline



Common Retrieval Performance Tuning Techniques

1. Tune Top-K

One of the easiest optimizations.
Bad
results = index.query(
 vector=query_embedding,
 top_k=100
)
Too many documents.
Better
results = index.query(
 vector=query_embedding,
 top_k=5
)
Now only the best documents are returned.
How to choose?
Tip-K Effect
1 May miss important context
3 Good for FAQs
5 Common choice
10 Large enterprise systems
100 Too much noise



2. Improve Chunk Size

Bad
5000 word chunk
Too much information.
Too small
20 words



2. Improve Chunk Size ★★★★★

Bad
5000 word chunk
Too much information.

Too small
20 words
Not enough context.

Good
100-500 words

or
500-1000 characters

depending on the embedding model and document type.

Example
RecursiveCharacterTextSplitter(
 chunk_size=500,
 chunk_overlap=100
)

3. Chunk Overlap

Without overlap
Chunk1
Machine Learning is...
Chunk2
...used in AI
Information gets split.

With overlap
Chunk1
Machine Learning is...
Chunk2
Learning is used in AI
Better retrieval.

4. Hybrid Search ★★★★★

Instead of
Only BM25
Use
BM25 + Vector Search
Better recall.

5. Metadata Filtering

Instead of searching
10 Million Documents
Search only
1M Documents
Faster retrieval.
Example

```
filter=[  
  "Department": ["  
    "Seq""HR"  
  ]  
]
```

6. Query Rewriting

Original
How does it work?
Rewrite
How does Pinecone Hybrid Search work?
Much better retrieval.

7. Re-ranking ★★★★★

Retriever returns
20 documents
Re-ranker selects
Top 5
Higher quality context.

8. Better Embedding Model

Instead of
Small embedding model
Use
Large embedding model
Semantic retrieval improves.

9. Normalize Embeddings

embedding = embedding / np.linalg.norm(embedding)
Improves cosine similarity consistency.

10. Remove Duplicate Chunks

Retriever
Chunk1
Chunk2
Chunk2
Chunk3
Remove duplicates
Chunk1
Chunk2
Chunk3
More useful context.

11. Multi-query Retrieval

Instead of searching
What is Hybrid Search?
Generate
Explain Hybrid Search
BM25 + Vector Search
Semantic Search
Retrieve using all queries.
Merge results.
This improves recall.

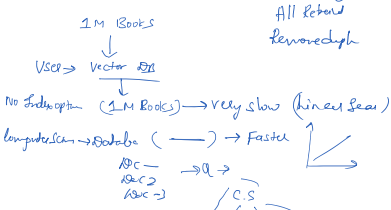
12. Index Optimization

Choose the appropriate vector index.

- Examples
- HNSW
 - IVF
 - Flat Index

Each offers different trade-offs between speed, memory, and accuracy.

Production Retrieval Pipeline



Index Optimization

What is Index Optimization?

Definition
Index Optimization is the process of choosing and configuring the most suitable indexing algorithms and parameters to improve the speed, memory usage, and retrieval accuracy of a vector database.

In simple words:
Instead of checking every vector in the database, the index organizes vectors so that similar vectors can be found much faster.

Why Do We Need an Index?

Suppose you have 1 million document embeddings.

User asks
What is Hybrid Search?

Without an index
Query Vector

↓
Compare with Doc1
Compare with Doc2
Compare with Doc3
Compare with Doc1000000
This is called **Brute Force Search**.
Time Complexity
O(N)
If there are
1 Million vectors
you compare against all one million vectors.
Very slow.

With Index

Instead
Query Vector
↓
Vector Index
↓
Nearest Neighbors
Now only a few thousand vectors are examined.
Much faster.

Real-Life Analogy

Imagine a library with 10 lakh books.

Without index
Find Python Book

↓
Check every shelf
May take hours.

With index
Computer

↓
Shelf 12

↓
Row 3

↓
Book Found

Much faster.

What is LangChain?

The query is converted into an embedding.

Now the database must find the most similar embedding.

Brute Force Search

Query

↓
Compare with Vector 1

Compare with Vector 2

Compare with Vector 3

Compare with Vector 4

Compare with Vector 5

Compare with Vector 6

Compare with Vector 7

Compare with Vector 8

Compare with Vector 9

Compare with Vector 10

Compare with Vector 11

Compare with Vector 12

Compare with Vector 13

Compare with Vector 14

Compare with Vector 15

Compare with Vector 16

Compare with Vector 17

Compare with Vector 18

Compare with Vector 19

Compare with Vector 20

Compare with Vector 21

Compare with Vector 22

Compare with Vector 23

Compare with Vector 24

Compare with Vector 25

Compare with Vector 26

Compare with Vector 27

Compare with Vector 28

Compare with Vector 29

Compare with Vector 30

Compare with Vector 31

Compare with Vector 32

Compare with Vector 33

Compare with Vector 34

Compare with Vector 35

Compare with Vector 36

Compare with Vector 37

Compare with Vector 38

Compare with Vector 39

Compare with Vector 40

Compare with Vector 41

Compare with Vector 42

Compare with Vector 43

Compare with Vector 44

Compare with Vector 45

Compare with Vector 46

Compare with Vector 47

Compare with Vector 48

Compare with Vector 49

Compare with Vector 50

Compare with Vector 51

Compare with Vector 52

Compare with Vector 53

Compare with Vector 54

Compare with Vector 55

Compare with Vector 56

Compare with Vector 57

Compare with Vector 58

Compare with Vector 59

Compare with Vector 60

Compare with Vector 61

Compare with Vector 62

Compare with Vector 63

Compare with Vector 64

Compare with Vector 65

Compare with Vector 66

Compare with Vector 67

Compare with Vector 68

Compare with Vector 69

Compare with Vector 70

Compare with Vector 71

Compare with Vector 72

Compare with Vector 73

Compare with Vector 74

Compare with Vector 75

Compare with Vector 76

Compare with Vector 77

Compare with Vector 78

Compare with Vector 79

Compare with Vector 80

Compare with Vector 81

Compare with Vector 82

Compare with Vector 83

Compare with Vector 84

Compare with Vector 85

Compare with Vector 86

Compare with Vector 87

Compare with Vector 88

Compare with Vector 89

Compare with Vector 90

Compare with Vector 91

Compare with Vector 92

Compare with Vector 93

Compare with Vector 94

Compare with Vector 95

Compare with Vector 96

Compare with Vector 97

Compare with Vector 98

Compare with Vector 99

Compare with Vector 100

Compare with Vector 101

Compare with Vector 102

Compare with Vector 103

Compare with Vector 104

Compare with Vector 105

Compare with Vector 106

Compare with Vector 107

Compare with Vector 108

Compare with Vector 109

Compare with Vector 110

Compare with Vector 111

Compare with Vector 112

Compare with Vector 113

Compare with Vector 114

Compare with Vector 115

Compare with Vector 116

Compare with Vector 117

Compare with Vector 118

Compare with Vector 119

Compare with Vector 120

Compare with Vector 121

Compare with Vector 122

Compare with Vector 123

Compare with Vector 124

Compare with Vector 125

Compare with Vector 126

Compare with Vector 127

Compare with Vector 128

Compare with Vector 129

Compare with Vector 130

Compare with Vector 131

Compare with Vector 132

Compare with Vector 133

Compare with Vector 134

Compare with Vector 135

Compare with Vector 136

Compare with Vector 137

Compare with Vector 138

Compare with Vector 139

Compare with Vector 140

Compare with Vector 141

Compare with Vector 142

Compare with Vector 143

Compare with Vector 144

Compare with Vector 145

Compare with Vector 146

Compare with Vector 147

Compare with Vector 148

Compare with Vector 149

Compare with Vector 150

Compare with Vector 151

Compare with Vector 152

Compare with Vector 153

Compare with Vector 154

Compare with Vector 155

Compare with Vector 156

Compare with Vector 157

Compare with Vector 158

Compare with Vector 159

Compare with Vector 160

Compare with Vector 161

Compare with Vector 162

Compare with Vector 163

Compare with Vector 164

Compare with Vector 165

Compare with Vector 166

Compare with Vector 167

Compare with Vector 168

Compare with Vector 169

Compare with Vector 170

Compare with Vector 171

Compare with Vector 172

Compare with Vector 173

Compare with Vector 174

Compare with Vector 175

Compare with Vector 176

Compare with Vector 177

Compare with Vector 178

Compare with Vector 179

Compare with Vector 180

Compare with Vector 181

Compare with Vector 182

Compare with Vector 183

Compare with Vector 184

Compare with Vector 185

Compare with Vector 186

Compare with Vector 187

Compare with Vector 188

Compare with Vector 189

Compare with Vector 190

Compare with Vector 191

Compare with Vector 192

Compare with Vector 193

Compare with Vector 194

Compare with Vector 195

Compare with Vector 196

Compare with Vector 197

Compare with Vector 198

Compare with Vector 199

Compare with Vector 200

Compare with Vector 201

Compare with Vector 202

Compare with Vector 203

Compare with Vector 204

Compare with Vector 205

Compare with Vector 206

Compare with Vector 207

Compare with Vector 208

Compare with Vector 209

Compare with Vector 210

Compare with Vector 211

Compare with Vector 212

Compare with Vector 213

Compare with Vector 214

Compare with Vector 215

Compare with Vector 216

Compare with Vector 217

Compare with Vector 218

Compare with Vector 219

Compare with Vector 220

Compare with Vector 221

Compare with Vector 222

Compare with Vector 223

Compare with Vector 224

Compare with Vector 225

Compare with Vector 226

Compare with Vector 227

Compare with Vector 228

Compare with Vector 229

Compare with Vector 230

Compare with Vector 231

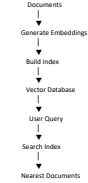
Compare with Vector 232

Compare with Vector 233

Compare with Vector 234

BOOK round
Much faster.

Architecture



Types of Vector Indexes

1. Flat Index (Brute Force)

Query
↓
Compare with every vector
Example
Q
↓
V1
V2
V3
V4
V5
V1000000
↓
Advantages
☒ 100% accurate
Disadvantages
☒ Very slow

2. HNSW (Hierarchical Navigable Small World)

Most popular vector index.
Used by
• Pinterest
• Giphy
• Weibo
• Milvus
Idea
Instead of checking every vector
Query
↓
Closest Cluster
↓
Closest Node
↓
Nearest Documents
Like Google Maps.
You don't visit every road.
You follow the best roads.

Advantages
☒ Very fast
☒ Very accurate

Disadvantages
Uses more memory.

3. IVF (Inverted File Index)

Idea
First cluster vectors.
All Documents
↓
Cluster 1
Cluster 2
Cluster 3
Cluster 4
When user searches
Query
↓
Nearest Cluster
↓
Search only that cluster
Instead of
1. Million vectors
search
10,000 vectors

Advantages
Fast
Less memory

Disadvantages
May miss some nearest neighbors if the correct vector lies in another cluster.

Top-4 Documents

A user asks:

"What is LangChain?"
The query is converted into an embedding.
Now the database must find the most similar embedding.

Brute Force Search

Query
↓
Compare with Vector 1 ✗
Compare with Vector 2 ✗
Compare with Vector 3 ✗
...
Compare with Vector 10,000,000 ✗
↓

So we need an indexing technique.
There are many indexing techniques.
The most popular are:
• HNSW
• IVF

HNSW (Graph-Based Search)

"Imagine you are in Bangalore and want to reach Mysore."
Will you check every road in Karnataka?
No.
You'll follow Google Maps.
Google Maps helps taking you through the **best nearby roads** until you reach the destination.
HNSW works exactly like that.
Instead of comparing with every vector,
it jumps from one similar vector to another.

Query
↓

Vector A
↓

Vector C
↓

Vector F
↓

Vector K
↓

Best Match
It never checks every vector.
It only travels through connected vectors.

One-line definition

HNSW stores vectors as a graph and finds the nearest vector by navigating through neighboring vectors.

IVF (Cluster-Based Search)

Suppose your college library has
1 million books.

Books are arranged like this:

Python
Java
↓
Networking
History
↓
Science
↓
Student asks

"Python Basics"

Will the librarian search every book?
No.

They first go to the **Python section**.

Then search inside that section.

IVF does exactly this.

First it divides vectors into clusters.

Cluster 1

Cluster 2

Cluster 3

Cluster 4

Cluster 5

When a query comes,
it first finds the nearest cluster.

Query
↓

Nearest Cluster
↓

Search only inside that cluster
↓

Answer
Instead of searching 10 million vectors,
it searches maybe only 10,000 vectors.

One-line definition

IVF groups similar vectors into clusters and searches only the closest cluster(s).

Visual Comparison

Without index
Query
↓

10 Million Vectors
↓

Answer

HNSW

Query
↓

Jump → Jump → Jump
↓

Best Match

IVF

Query
↓

Find Cluster
↓

Search inside Cluster
↓

Best Match

Difference

HNSW

Graph-Based

Moves through neighboring vectors

Higher accuracy

More memory

IVF

Cluster-based

Searches inside relevant clusters

Faster indexing

Less memory

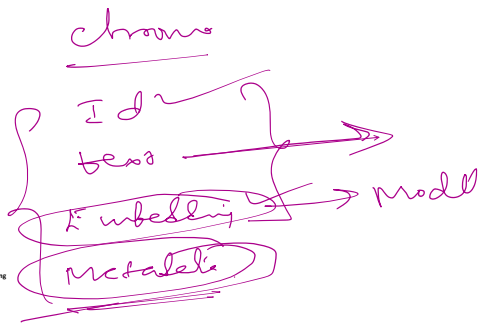
Interview Question

Interviewer: "When would you choose HNSW over IVF?"

Answer:

"If I need very high search accuracy and low latency, I'll use HNSW. If I have an extremely large dataset and want lower memory usage with faster indexing, I'll use IVF."

Multi-similarity



Doc 1

Python is pgn de

[0.01, -0.112, 0.33]

{ source → pdf, data }

Get all table name

['employee'] → each → 1

(dept),
(paycol)

[row[0] for row in Result]

table = ['employee', 'dept']

table = []

for row in result:
table.append(row)

-value

]

' " pu]